



Mastering TestNG

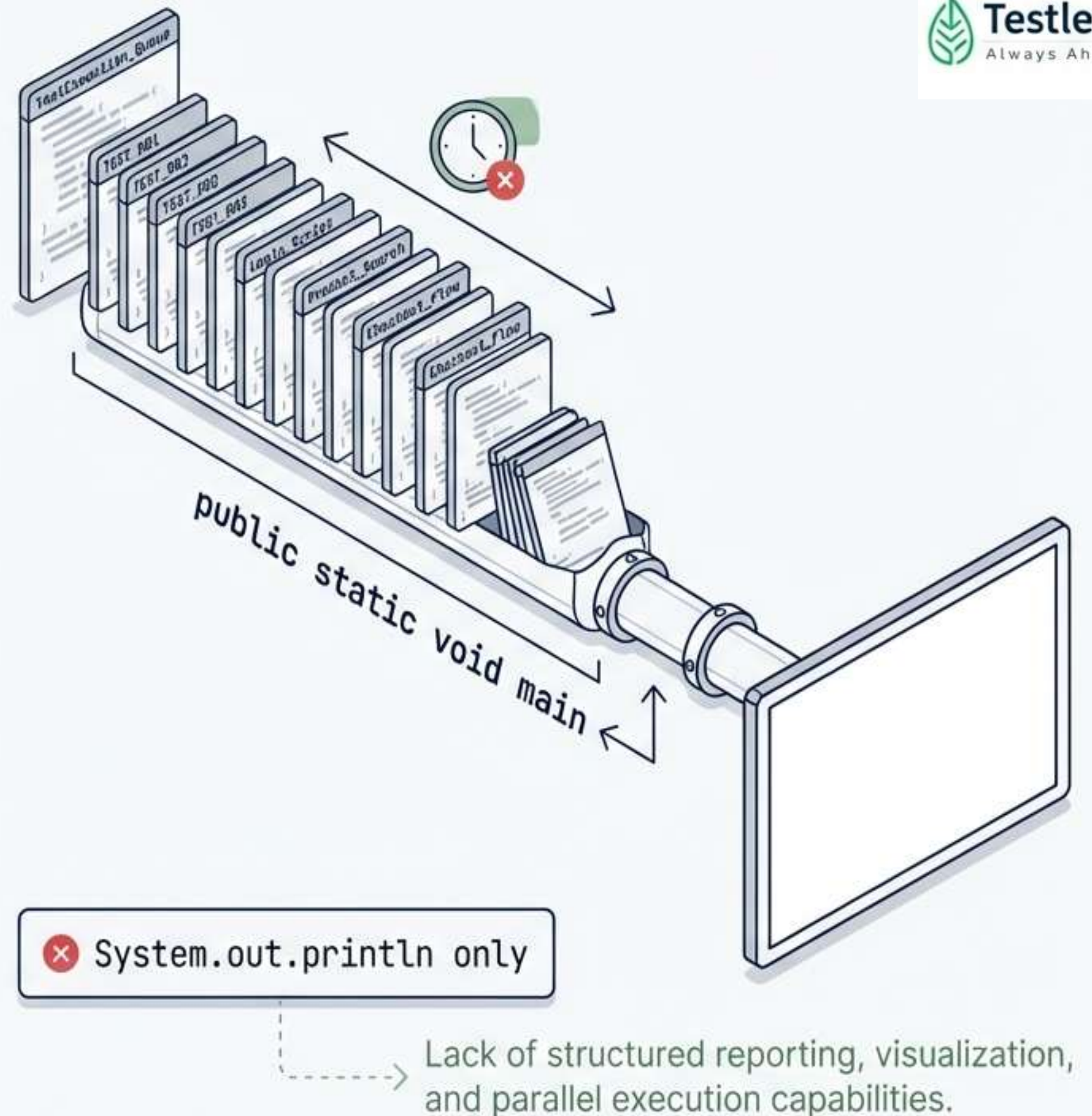
The Framework for Next-Generation Testing

FROM MANUAL EXECUTION TO SCALABLE TEST ARCHITECTURE

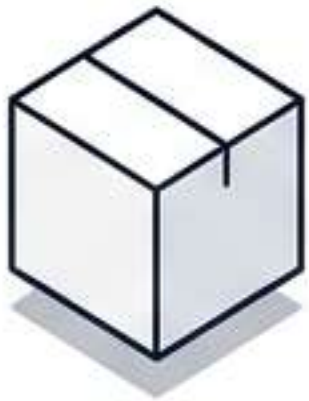
The Limitation of the Main Method

In standard automation, relying on the Java `main()` method creates significant bottlenecks for Quality Assurance.

- ❌ **Static Execution**
Code is rigid and difficult to manage as complexity grows.
- ❌ **Zero Visibility**
No built-in reporting or execution status feedback.
- ❌ **Sequential Bottleneck**
Tests run linearly, ignoring multi-core capabilities.



Enter TestNG: The Open Source Powerhouse



WHAT

An open-source unit testing framework designed to cover all categories of tests, replacing the need for static main methods.



WHY

To gain robust reporting, parallel execution capabilities, and flexible configuration unavailable in standard Java.



HOW

By integrating the TestNG plugin and dependencies directly into the project structure.

TestNG = Test Next Generation

Key Capabilities of the Framework



Execution Control

Supports multiple execution in a single run, both sequentially and in parallel.



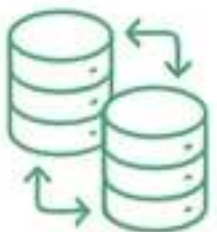
Reporting

Automatically generates reports providing the precise status of execution.



Configuration

Extensive control using annotations and attributes.



Data Handling

Supports multiple sets of test data (Data-Driven Testing).



Reliability

Includes Assertion support and automatic rerunning of failed test cases.



Categorization

Organize and filter test cases using TestNG groups.

Refactoring Strategy: Java to TestNG

Standard Java

```
public class Login {  
  
    public static void main(String[] args) {  
        // Test logic  
    }  
}
```

1.
Strip `static`, `main`,
and `args`.

2.
Rename to a
meaningful method.

TestNG

```
import org.testng.annotations.Test;  
  
public class Login {  
    @Test  
    public void runLogin() {  
        // Test logic  
    }  
}
```

3.
Apply @Test
annotation.

4.
Import
`org.testng.annotations`.

The Blueprint: Understanding testng.xml

The `testng.xml` file is the control room of your automation suite. It decouples execution logic from test code.



Root configuration for the entire suite.



Controls **how** tests are run (Parallel vs. Sequential).

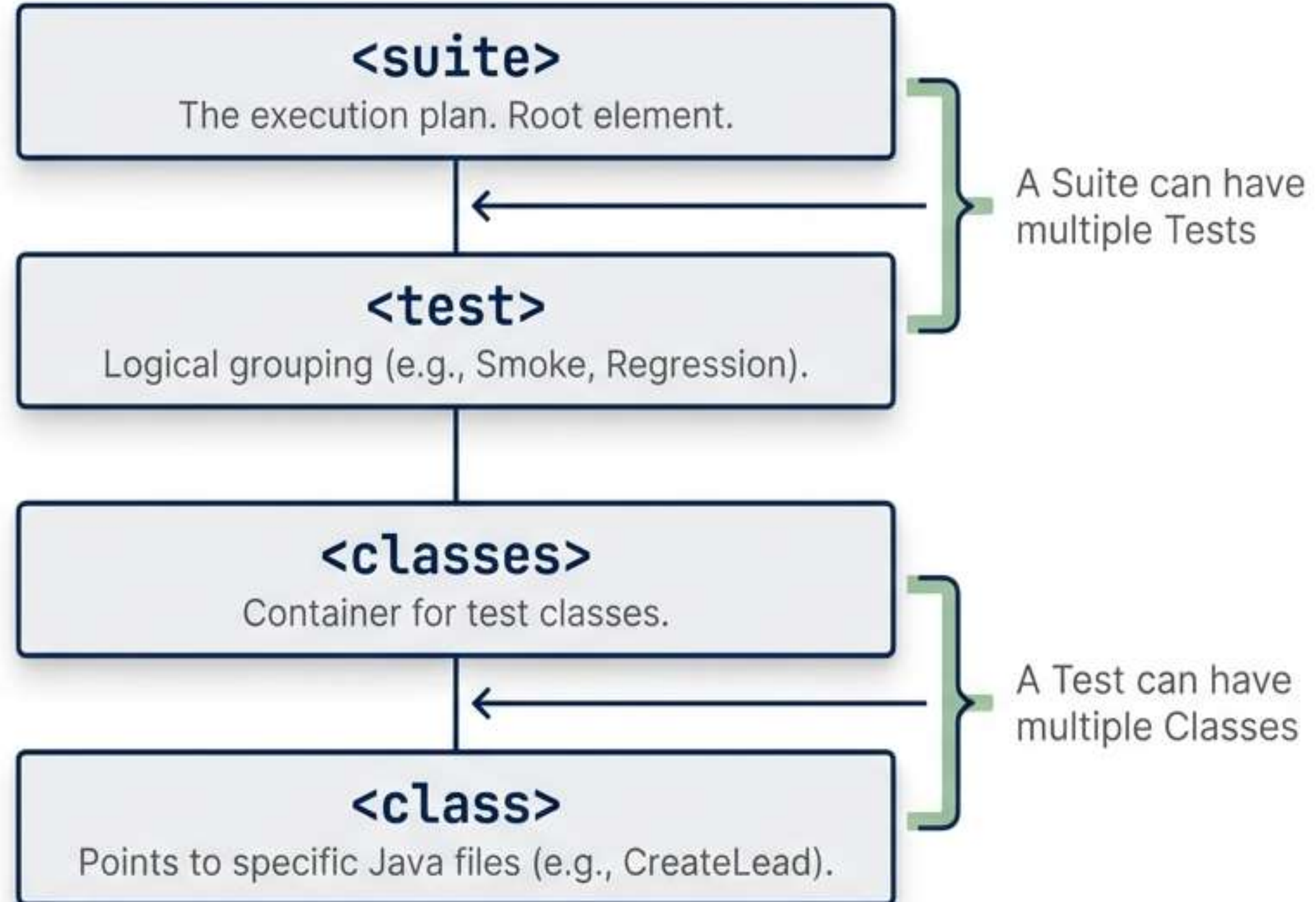


Allows precise scope definition (Suites, Tests, Classes).

```
1  <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"
2  <suite name="Suite" parallel="tests">
3      <test thread-count="5" name="Test" verbose="5">
4          <classes>
5              <class name="testcase.CreateLead" />
6              <class name="testcase.DeleteLead" />
7          </classes>
8      </test>
9  </suite>
```

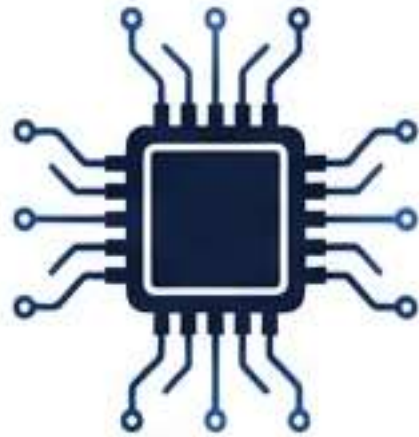
The Control **Blueprint**

XML Suite



Fine-Tuning Execution Attributes

Technical Editorial | Architectural Blueprint



thread-count

Controls the number of threads used during execution. Essential for managing concurrency load.

```
thread-count="5"
```



name

Allows customized naming for the Suite or Test (e.g., 'Smoke Test'), which appears in final reports.

```
name="Regression"
```

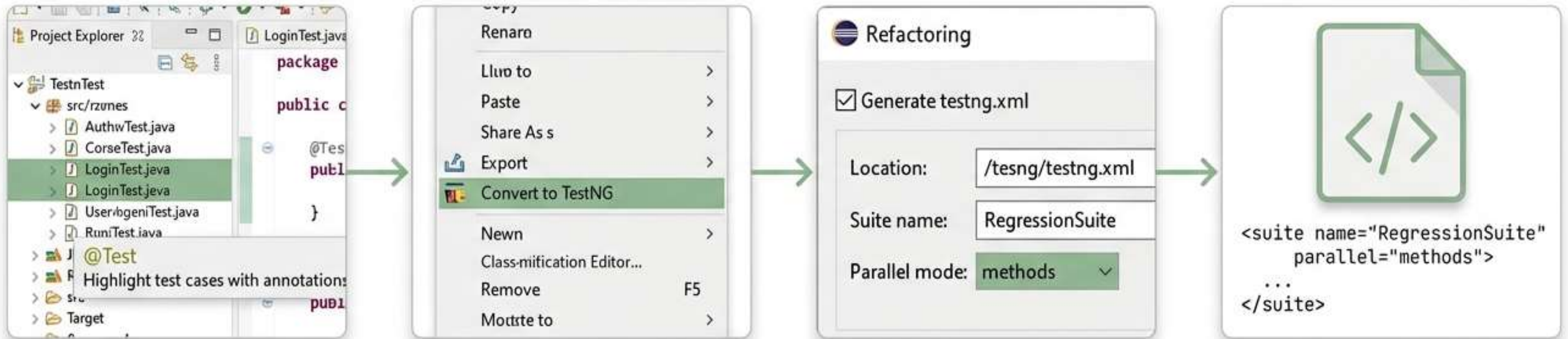


verbose

Controls the level of log detail in the console. Higher numbers = more debug information.

```
verbose="5"
```


Workflow: Generating the Execution Suite



Select

Highlight test cases with annotations.

Convert

Right-click -> Convert to TestNG.

Generate

Check configuration in the setup dialog.

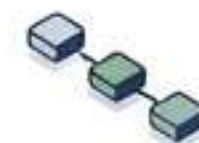
Configure

Finalize suite names and parallel modes.

```
<suite name="RegressionSuite"
parallel="methods">
...
</suite>
```

Breaking the Speed Limit: Parallel Execution

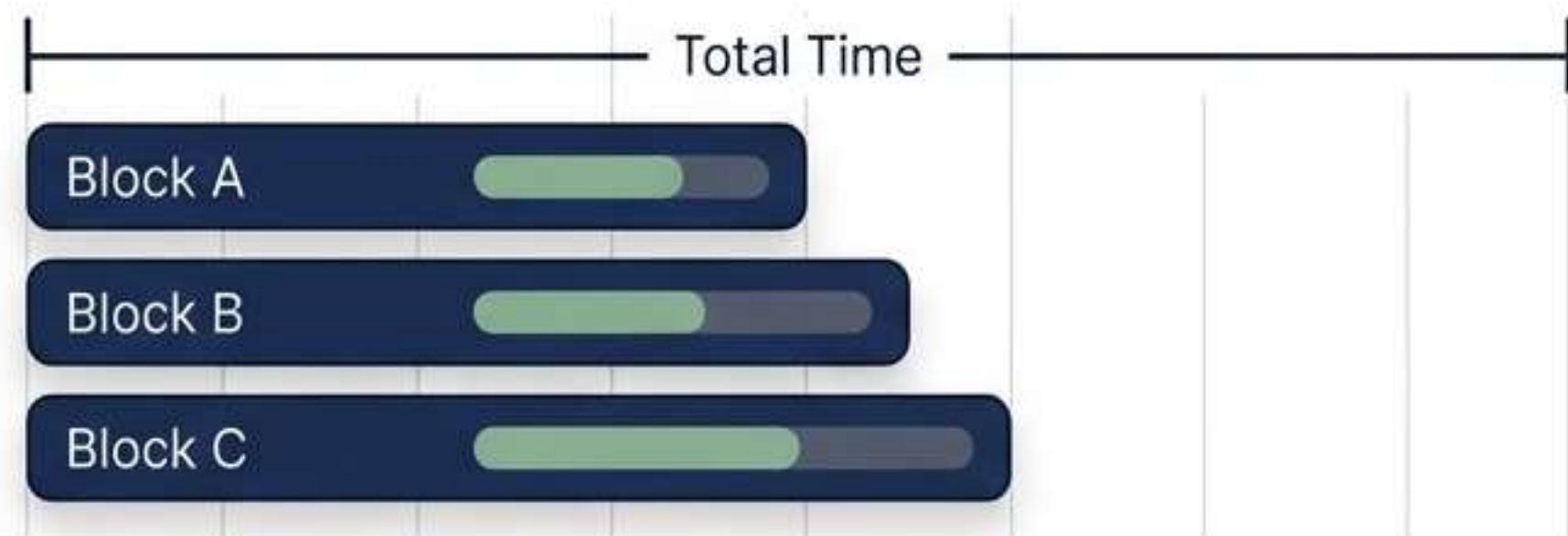
Sequential Execution (Default)



`mode="none"`

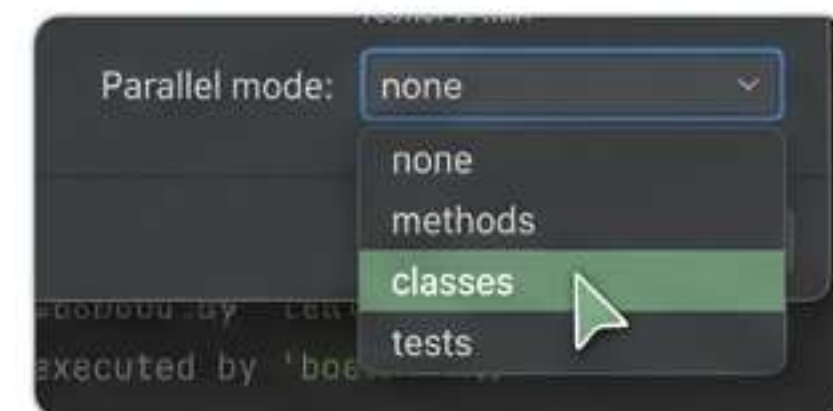
Tests run one after another.
Time accumulates linearly.

Parallel Execution (Optimized)



`parallel="classes"`

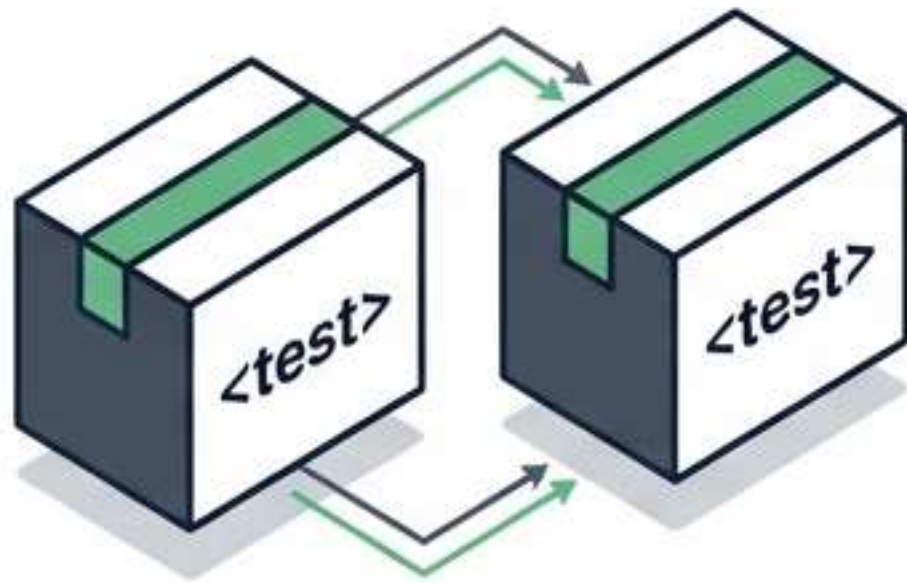
Splits execution across multiple
threads, drastically reducing
build time.



Three Modes of Parallelism

Tests

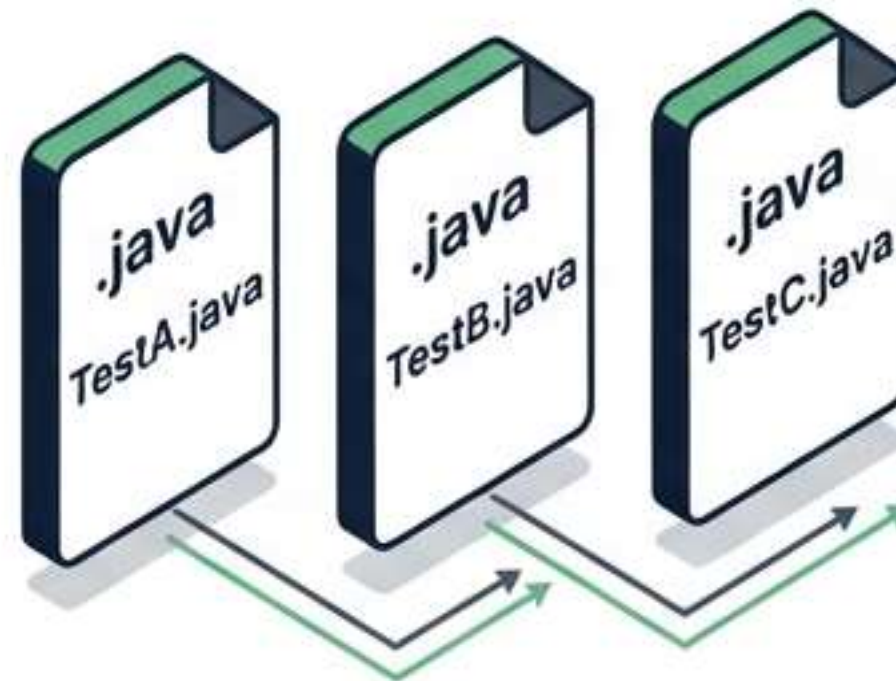
`parallel="tests"`



Runs multiple <test> tags (suites) simultaneously. Ideal for multi-environment execution.

Classes

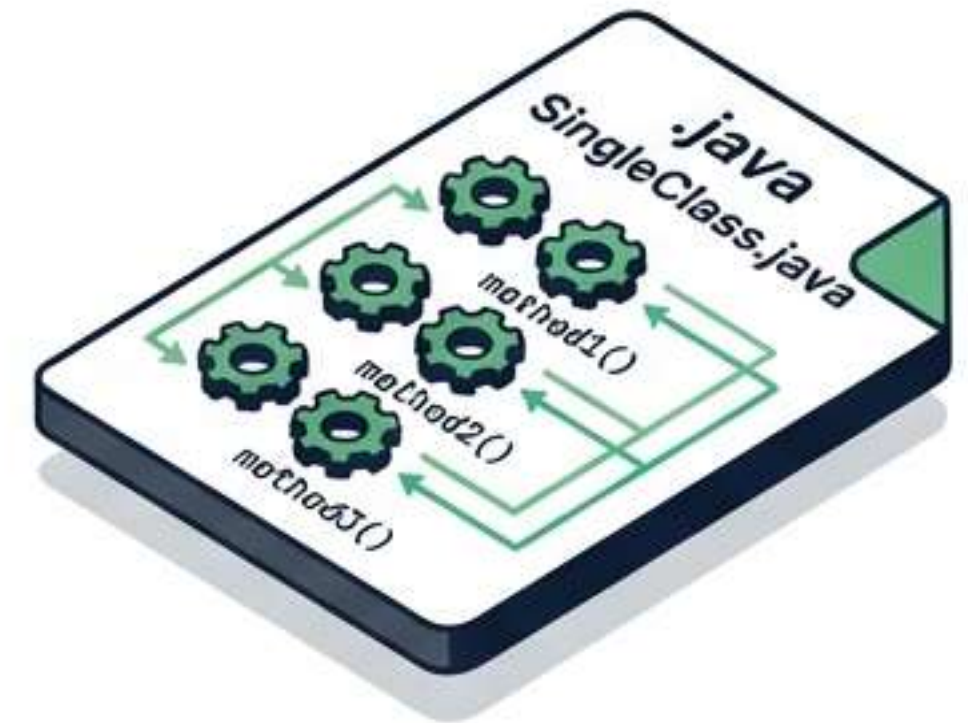
`parallel="classes"`



Runs multiple class files in parallel. The most common strategy to speed up a suite.

Methods

`parallel="methods"`



Runs individual methods inside a single class in parallel. Maximum granularity.

The Architect's Takeaway

Summary of Features & Benefits

Comprehensive Reporting

TestNG generates detailed logs and status reports automatically.

Scalability

Handles multiple datasets and executes massive test suites in a single run.

Flexible Execution

Switch seamlessly between Sequential debugging and Parallel performance modes.

Diagnostics

Fine-tune console output depth using the ``verbose`` attribute.

